

Tools for software development

Automated testing



Automating testing

- the basic reason **"increasing efficiency"**
- when testing without using full automation:
 - automating test data generation,
 - export and transformation of input data,
 - creating scripts and simple tools,
 - comparing results, e.g. in an Excel spreadsheet

Automating testing

- Benefits of introducing automation into testing:
 - significant reduction in maintenance costs thanks to regression test automation
 - high regression test coverage due to increased number of automated cases
 - increasing the performance of testers and analysts by participating in automation
 - achieving higher efficiency of some tests
 - if a framework for automated tests is created, it can also be used in the future



Myths about automated testing

- *" is a panacea for all testing problems "*
- *" by introducing automation, we will reduce the number of testers "*
 - the main goal is to increase the efficiency and quality of tests
- *" we automate 100% of cases for better results "*
- *"automation pays off immediately"*
 - initial development costs are only recovered months later
- *" automation will solve our problems with testing organization "*
- *" automated tests will replace manual tests "*

“ Manual ” vs. automated

- the misconception is in direct substitution
- **Automated tests**
 - they perform the same action repeatedly, quickly and flawlessly
 - automatic data verification, completion check
 - script execution
- **“Manual” tests**
 - are performed with a certain degree of variability as determined by the person during the design
 - human factor – determination, e.g. the writing is illegible
 - processing non-standard inputs

Types of automated tests

- **regression tests** - verification of functional and non-functional requirements
- **performance testing**
 - generating load from hundreds to thousands of users
 - load generation during 24 hours
- **functional tests**
 - automatic determination of all test cases and their execution
 - extensive test procedure that must be repeated flawlessly
- **unit tests**
 - are in principle always performed automatically
- **safety tests**
 - verification of resistance to known and variant methods of attacking the system
 - performing automated unit /functional tests with the aim of breaching security

Automation techniques

- Various techniques can be used to implement it, focusing on:
 - modularity of generated scripts
 - sustainability
 - easy reusability
 - necessity of technical knowledge
- **Capture and replay user activity**
- **Modifying generated scripts**
- **Data-driven testing**
- **Keyword-driven testing**

Capture and replay user activity

- *record and playback*
- the most well-known form of automation
- has satisfactory results even without technical knowledge
- the testing tool captures the activity of the user executing test cases
- subsequent playback of saved tests is possible
- test evaluation – comparison of variables, expected value
- Disadvantages:
 - high sensitivity to any functional and technical changes to the application
 - all inputs are recorded as static values
 - low modularity
 - inability to handle error states and failures
- Appropriate technique in later stages of the project

Modifying generated scripts

- it is also based on the records of the tests performed
- thanks to knowledge of the programming language, we can make changes to scripts
- implementation option:
 - more complex logic
 - expanding the content of the test
 - variability of variables
 - better sustainability
 - reusability of the given script
- When recording data, a static value is stored:
 - recorded input – current date – 2 6 .1 0 .202 1
 - next day test not working
 - replacing a static value with a function to return the current date

Data-driven testing

- *data-driven testing*
- advantageous in situations where repeated tests are necessary
 - with the same logic
 - with large and variable volume of input/output data
- data source:
 - database
 - table
 - CSV, XLSX file
- The test reads the input data and compares it with the expected result (line by line)
- sometimes it is necessary to create your own framework

Data-driven testing

- input table for calculating attack damage in the game:

Inputs				Expected output	
Efficiency	Bonus	Happiness	Accuracy	Resulting damage	Own loss
10.5	2	1	10	31.5	7,875
12	12	1	3.5	20	5
0	5	4.4	10	0	0
0	5.6	1	45	0	0
14.1	0	5	4	39.1	9,775
8	9.2	0	80.3	89.3	22,325

Keyword-driven testing

- *keyword-driven test*
- they are similar in structure test-driven data
- they contain not only input data but also commands
 - they create a test script (keywords)
 - at runtime it is loaded and test logic is dynamically created
- **Keywords:**
 - actions in the user interface
 - Clicking the button
 - Entering a value
 - Value verification
 - a tester can easily put together a test case from them without knowledge of their implementation
 - high level of abstraction

Keyword-driven testing

Advantages of the technique:

- easy to use without any knowledge of scripting language required
- the ability to prepare automated tests in the early stages of development
- very good sustainability
- high resistance to change

Must have:

- available framework that will support it before testing begins
- support from the development team / experienced testers
 - If the keyword does not exist, it cannot be used.

Keyword-driven testing

- Sample input table:

Keyword	Parameter_1	Parameter_2	Description
User login	" name "	" password "	Logging into the application
String verification	Login.Message ...	" Logged in! "	Verifying successful login
Inserting a string	ObjectName ...	" Chain "	Search text
Click on the object	ObjectName ...		Start search
....			

Combined/hybrid approach

- most commonly used in practice
- Sooner or later, the current technology will become insufficient and must be changed.
- taking advantage of all techniques

Choosing a tool for automated testing

- The success of automated testing depends on choosing the right tool early enough.
- **Full technology support**
 - verification in a pilot project
 - mitigating the risk of unavailability of certain functionality
- **Tool price**
 - initial investment - long-term
 - price of expansion modules/licenses
- **Availability and size of support**
 - faster resolution of deficiencies with the tool

Choosing a tool for automated testing

Evaluation:

- Is the given tool sufficient from a test management perspective?
- What knowledge is needed for test case design?
- Possibility of sufficient modification and extensibility of test scripts?
- Sufficient test case management?
- Test Management Level – Creating Test Sets?
- Is it possible to run tests distributed across multiple computers?
- Does the tool have a sufficient level of analysis of results?

Prerequisite for implementing test automation

Requirements for the systematic introduction of automation:

- Automation should be handled by a specific team, identifying basic needs, planning a solution, and selecting a suitable tool.
- management support – planning, solutions
- enough time and financial resources
- appropriate strategy and detailed planning
- enough experienced people and appropriate tools
- emphasis on continuous maintenance of tests and tools

When performing tests:

- to be effective, the system must be sufficiently stable
- manual tests intended for automation must be correctly specified – clarity, sufficient details

What test cases can be automated?

- there is no correct answer

It is necessary to consider:

- stability as a basic criterion
 - test case frequently modified makes no sense
- frequency of execution
 - often and regularly
- importance
 - they verify critical functionality on every new build
- complexity of implementation
 - highly complex calculations, large number of operations, large volume of data, complexity of manual testing
- time-consuming
 - nighttime execution
- complexity of automation
 - simple automation – verification of numerical values
 - complex automation – correct display of graphics, colors

Performance testing

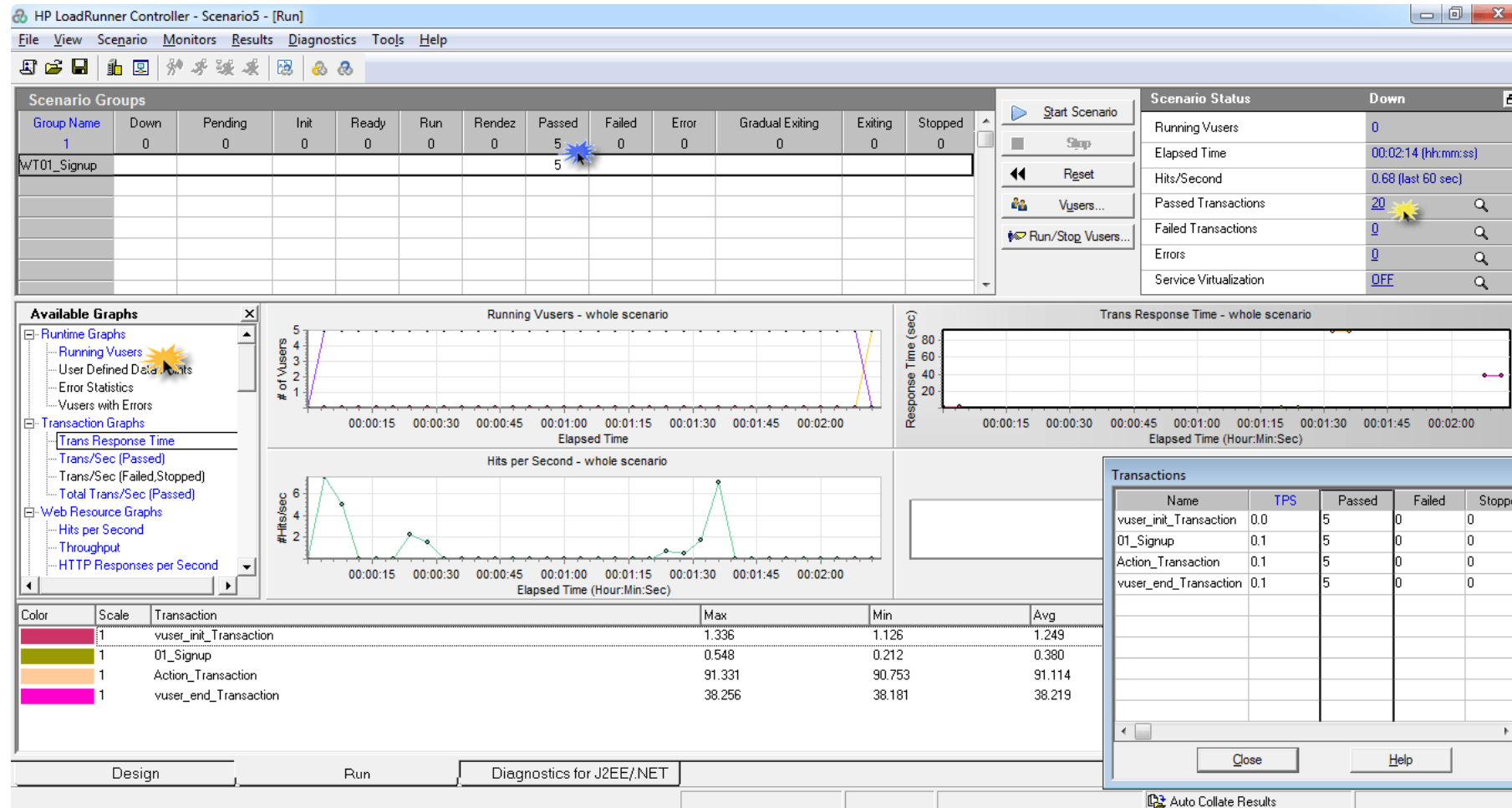
- The goal is to monitor system performance during scenarios with different load levels.
- do not require load generation
- comparison option

Performance measurement

- ***Response time***
 - system response time to a request
- **Availability**
 - time during which the system is available to the user
- **Throughput**
 - number of transactions performed at once
- **System resource usage**
 - CPU, memory, HDD load, ...

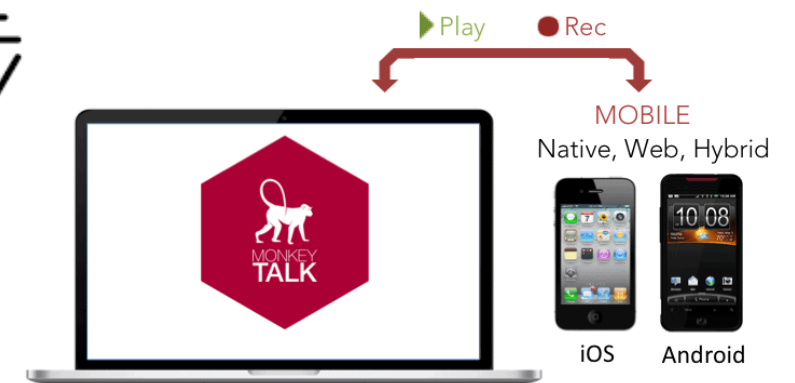
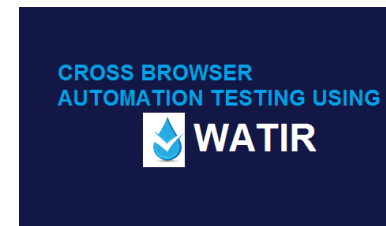
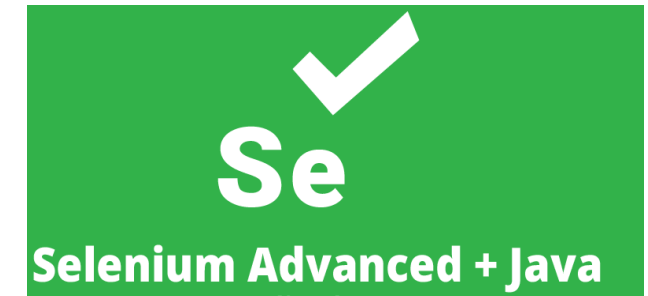
Limit load tests

- behavior under extreme stress and its impact

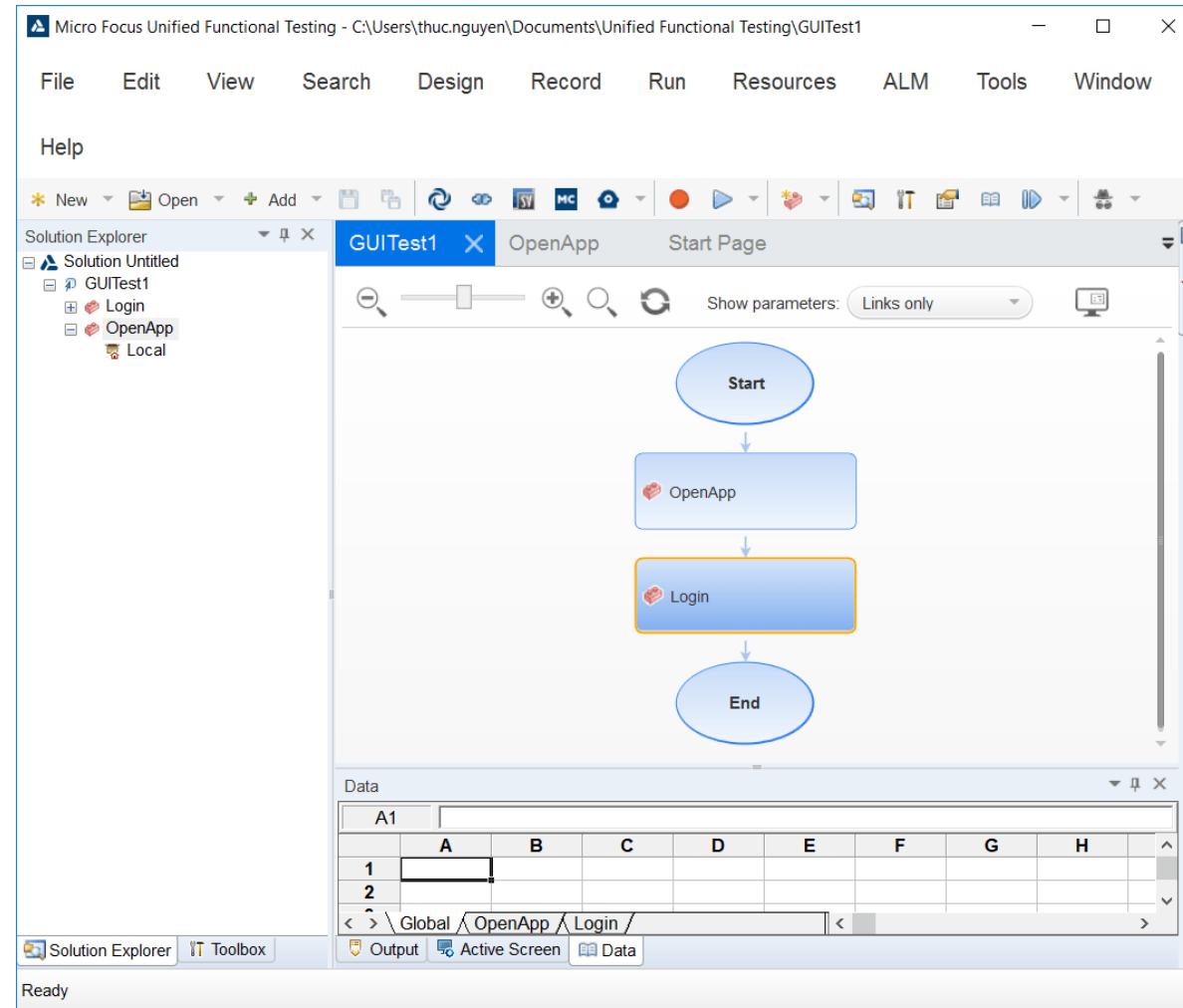


Automation software tools

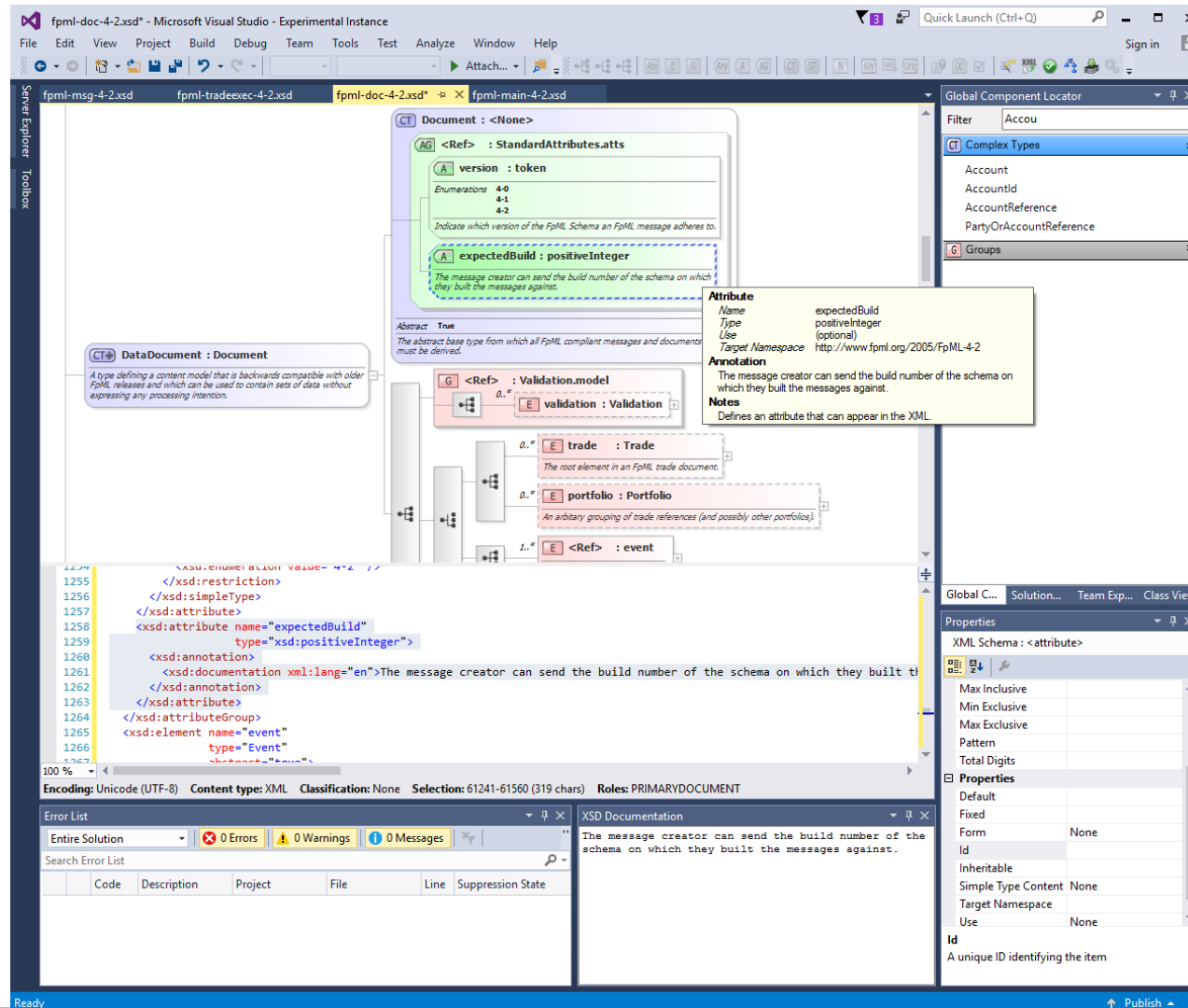
- Primary functional and regression tests:
- Paid:
 - Micro Focus Unified Functional Testing
 - Rational Functional Tester (IBM)
 - Visual Studio Test Professional
 - TestComplete - SmartBear Software
- Freely available:
 - Selenium
 - Watir
 - RobotFramework
 - MonkeyTalk – mobile applications



Micro Focus Unified Functional Testing



Visual Studio Test Professional



Selenium

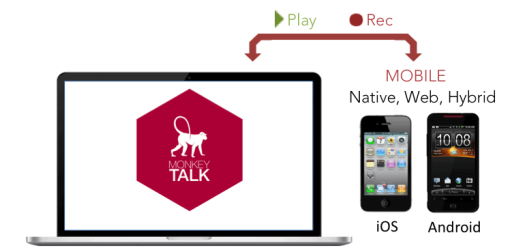


The screenshot shows the PyCharm IDE interface. On the left, the 'Project' tool window displays the file structure of 'MyProject' at 'C:\pythonSelenium\MyProject'. It includes a 'venv' directory with 'geckodriver.exe', 'geckodriver.log', and 'geckodriver-v0.20.0-win32.zip', along with 'PythonScript.py'. The 'External Libraries' section shows the Python 3.6 environment with various installed packages, including 'selenium' (version 3.11.0). The main editor window displays the code in 'PythonScript.py'. The code is a Selenium test script that uses the 'webdriver' module to interact with a Firefox browser. It navigates to 'https://www.google.com', finds a search input element by name 'q', clears it, and sends the text 'selenium' followed by a return key. It then uses 'implicitly_wait(45)' to wait for the results. A red arrow points to the 'implicitly_wait' line. The code concludes with an assertion to verify 'selenium' is in the page source, a print statement, and finally closes the driver.

```
1  # Your Selenium python test code goes here
2  # www.TestingDocs.com
3
4  +import ...
5
6
7  # Firefox driver
8  driver = webdriver.Firefox()
9  driver.get("https://www.google.com")
10
11  element = driver.find_element_by_name("q")
12  element.clear()
13  element.send_keys("selenium")
14  element.send_keys(Keys.RETURN)
15  # implicit wait
16  driver.implicitly_wait(45)
17  #Make sure you get some result with selenium
18  assert "selenium" in driver.page_source
19  print("Sample Test Completed.")
20  # close driver
21  driver.close()
```



MonkeyTalk



MonkeyTalk - newtest/test1.mt - MonkeyTalkIDE

File Edit Navigate Search Project Window Help

Project Explorer

- datadriven
 - JavaScript Resources
 - libs
 - datadriven.mt
- iterate
 - JavaScript Resources
 - libs
 - login.js
 - login.mt
- monkeytalk1
- newtest
 - ppp
 - JavaScript Resources
 - libs
 - rrr.mt

test1.mt result.mt rescredentials.mt test1.mt rrr.mt test.mt datadriven.mt

Row	Component	MonkeyID	Action	Arguments	Timeout (ms)	ThinkTime (ms)
1	Vars	*	Define	user pass		
2	Input	username	tap			
3	Input	username	EnterText	\$(user)		
4	Input	password	EnterText	\$(pass)		
5	Button	LOGIN	tap			3000
6	Button	LOGOUT	tap			3000

Table View MonkeyTalk JavaScript

Console JUnit

MonkeyTalk Output

```
15:45:58.924: Completed Script Playback - OK
15:46:27.310: Started Script Playback
15:46:27.318: Script test1.mt Run john "my password" %thinktime=3000
15:46:27.323: Started Script Playback
15:46:27.327: Vars * Define user pass
15:46:27.330: Input username tap
15:46:28.063: Input username EnterText john
15:46:28.842: Input password EnterText "my password"
15:46:29.446: Button LOGIN tap %thinktime=3000
15:46:32.601: Button LOGOUT tap %thinktime=3000
15:46:36.242: Completed Script Playback - OK
15:46:36.245: Script test1.mt Run john1 "my password1" %thinktime=3000
15:46:36.248: Started Script Playback
15:46:36.250: Vars * Define user pass
15:46:36.252: Input username tap
15:46:36.850: Input username EnterText john1
15:46:37.669: Input password EnterText "my password1"
15:46:38.312: Button LOGIN tap %thinktime=3000
15:46:41.484: Button LOGOUT tap %thinktime=3000
15:46:44.990: Completed Script Playback - OK
15:46:44.994: Completed Script Playback - OK
```

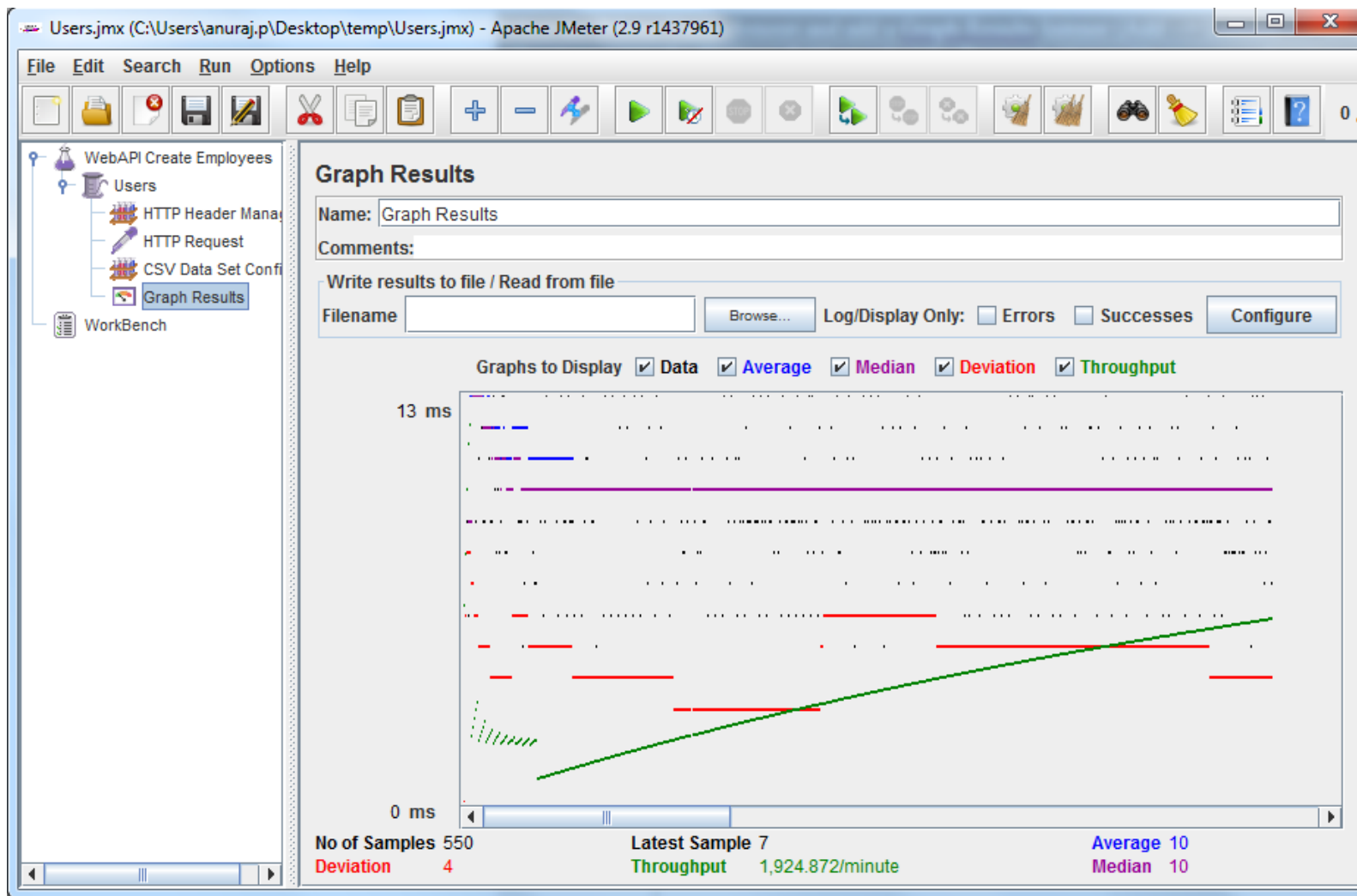


Automation software tools

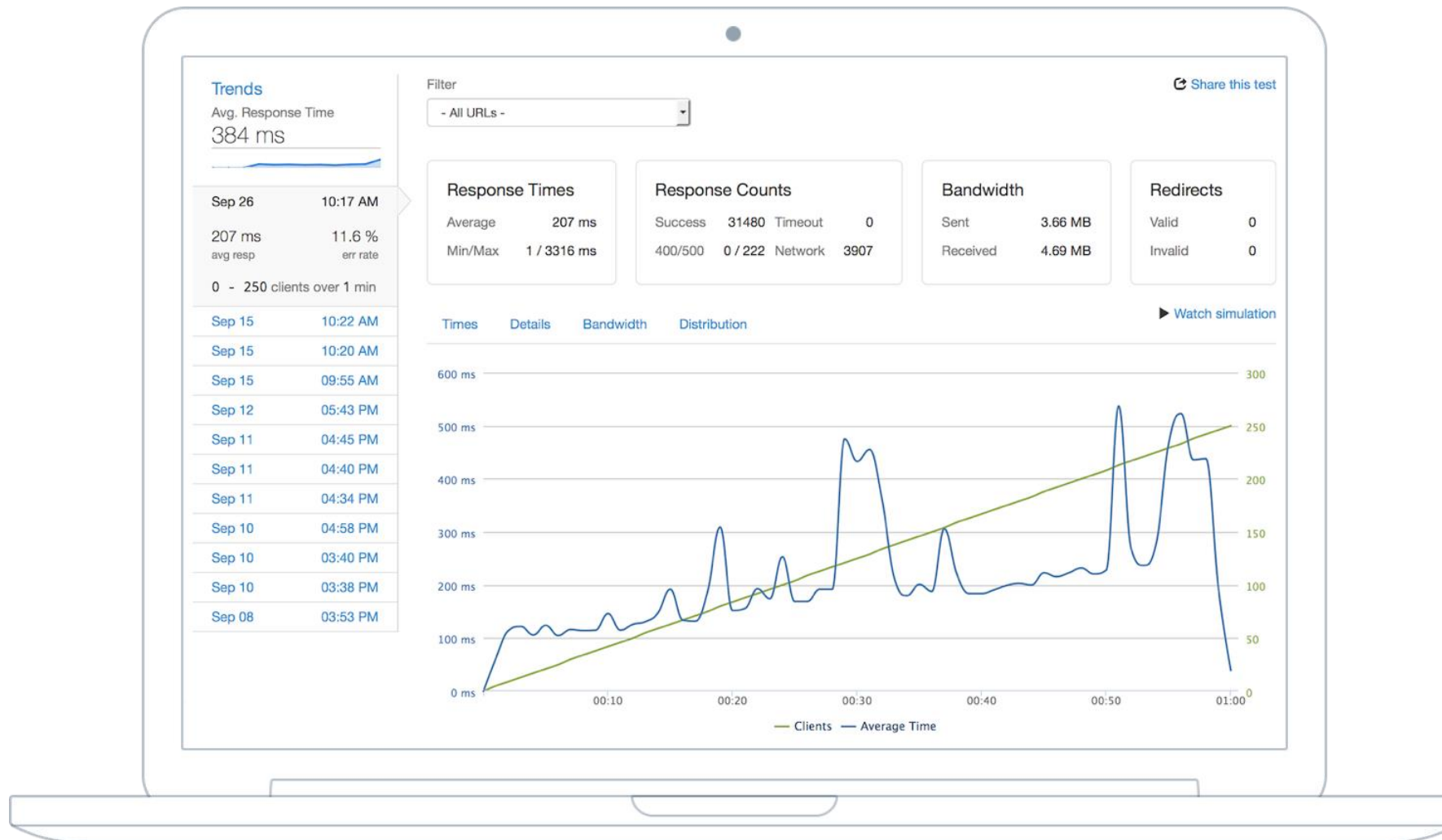
Performance tests:

- Paid
 - LoadRunner (HP)
 - Rational Performance Tester (IBM)
 - Silk Performer (Micro Focus)
- Freely available
 - Apache JMeter
 - Loader.io

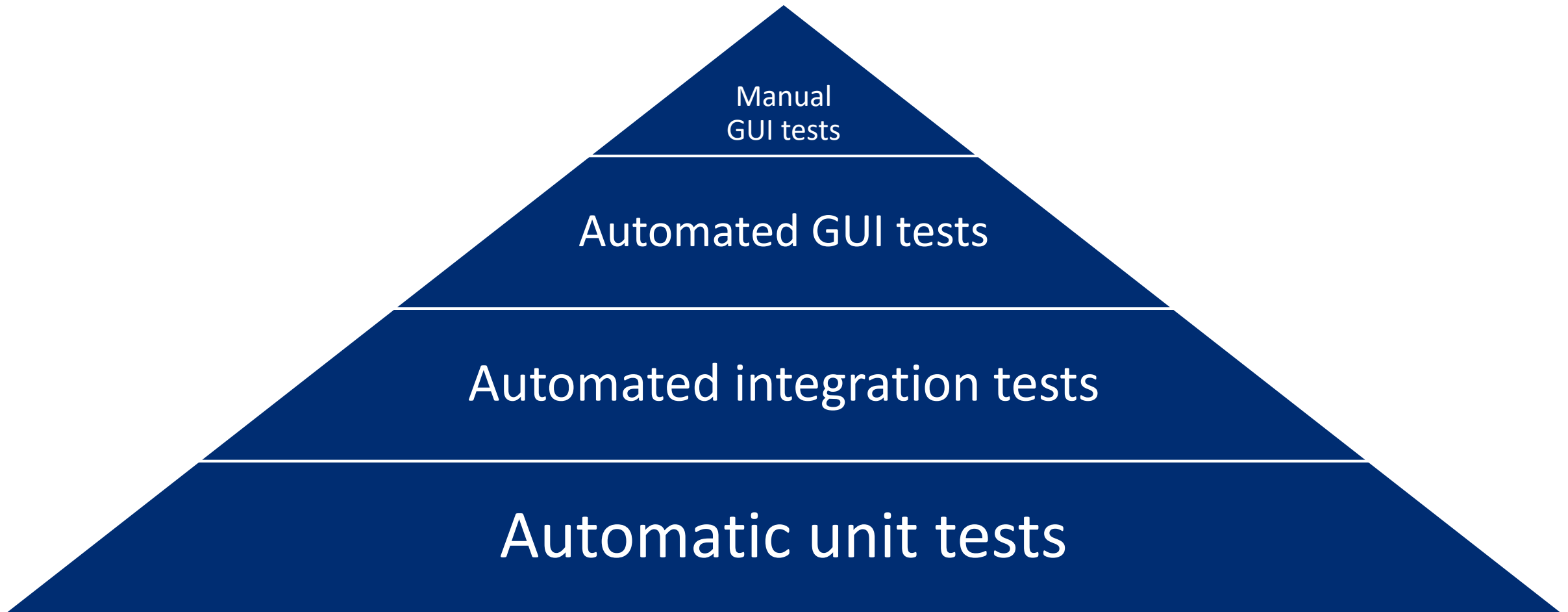
Apache JMeter



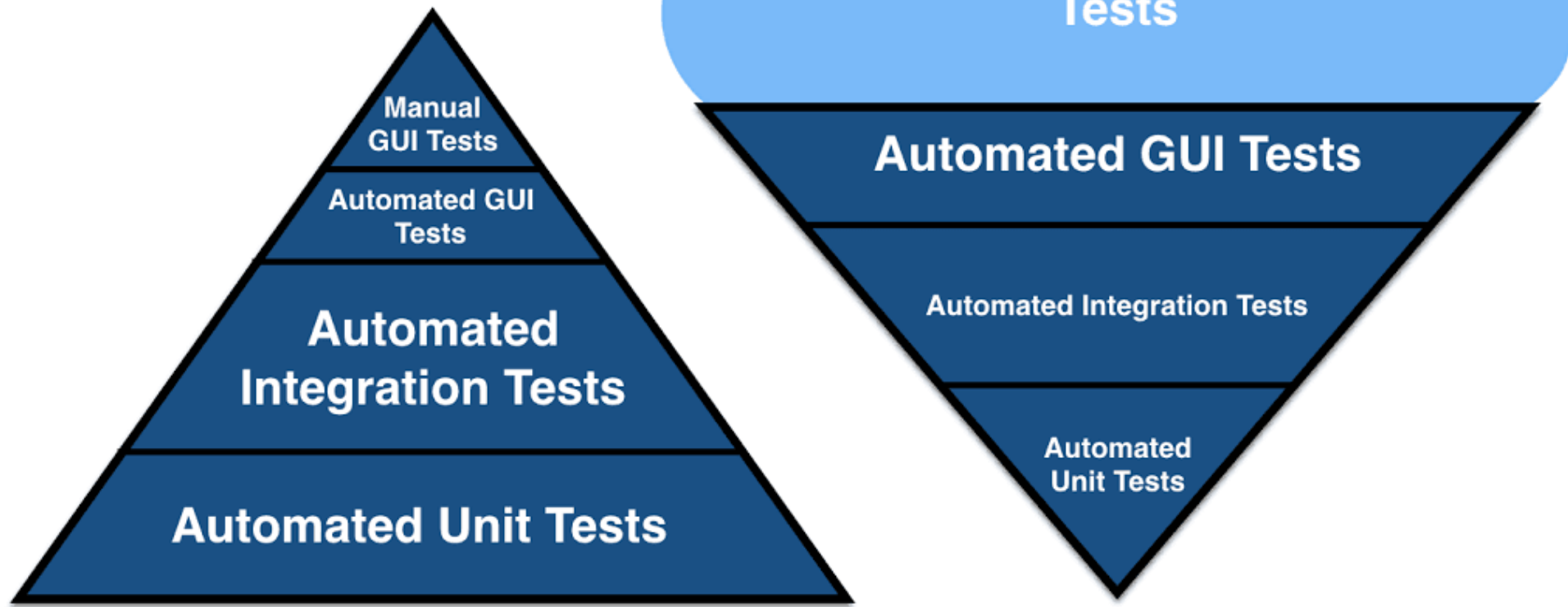
Loader.io

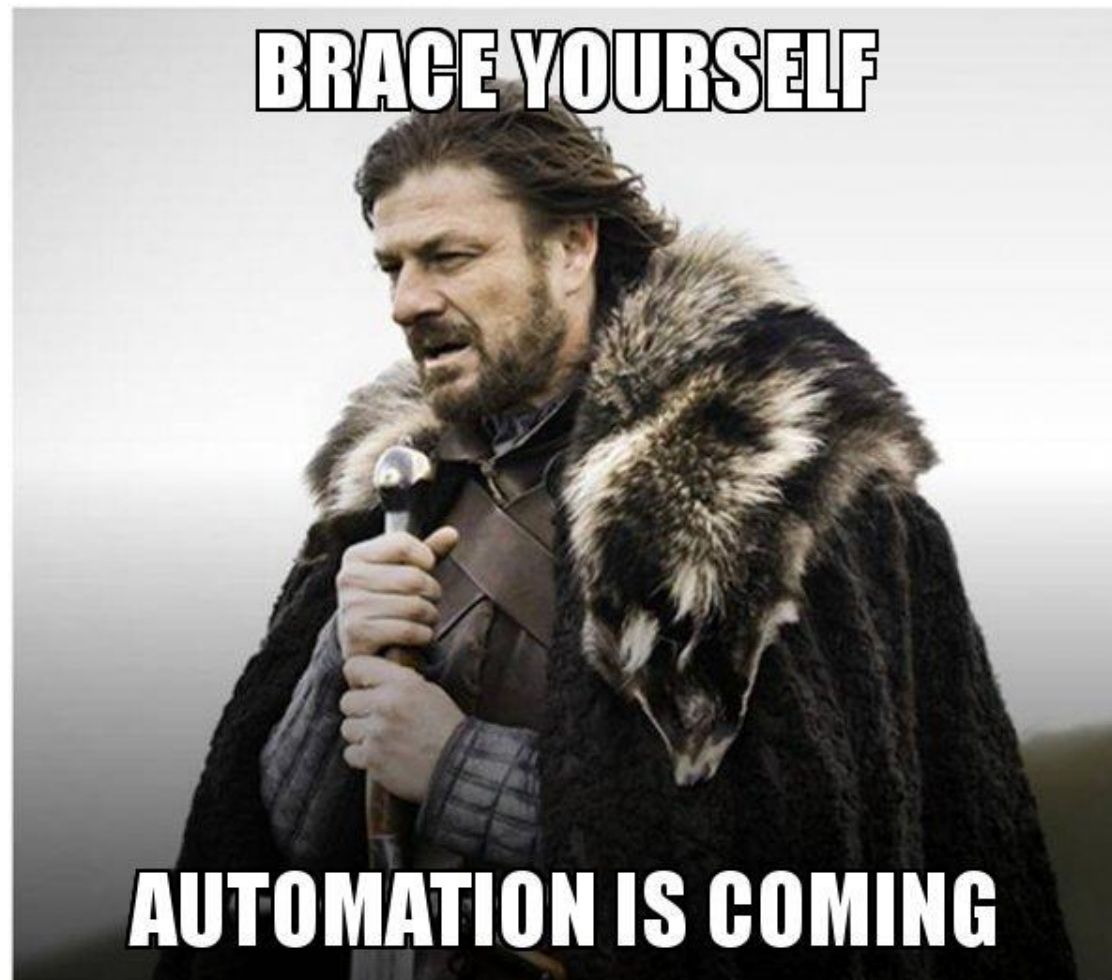


Planning strategy



Test pyramid vs Ice cream cone





Thank you for your attention

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk